Direct-Page Addressing for Relocatable Symbols: Design

mc6801-tools

November 2026

Contents

1 Summary **2**

2 Background: how it works today **2**

2.1 Choosing the addressing mode 2

2.2 Forced direct addressing of relocatable symbols 3

2.3 What else makes one-byte relocations 3

2.4 Imports carry a location counter 3

3 Goals and non-goals **4**

3.1 Goals 4

3.2 Non-goals 4

4 The DPAGE directive **4**

4.1 Forward references 5

4.2 Byte values 5

4.3 Choosing a counter 5

5 Linker checks **6**

6 Example **7**

7 Implementation **7**

7.1 1. ld01: check one-byte relocations 7

7.2 2. as01: DPAGE 8

7.3 3. Documentation 8

8 Testing **8**

9 Phases **9**

10 Decisions **9**

1 Summary

The MC6801's *direct* addressing mode reaches page zero (`$0000`–`$00FF`) with a one-byte address, so an instruction such as `ldaa count` takes 2 bytes and 3 cycles instead of the 3 bytes and 4 cycles of *extended* addressing. Page zero holds the chip's I/O registers and its 128 bytes of internal RAM, which is where a program's most used variables belong.

Today `as01` uses direct addressing only for addresses it knows when it assembles a module. A variable that a linker script places in page zero is relocatable, so every reference to it uses extended addressing unless the programmer writes `<` before it, and nothing checks that the variable really ends up in page zero.

This document proposes two changes:

1. **`ld01` checks every one-byte relocation.** A direct address, or an indexed offset, that doesn't fit in a byte is an error naming the module, the symbol and the address. Today such a link succeeds and silently produces wrong code. This is a bug fix on its own.
2. **A `DPAGE` directive in `as01`** marks a location counter as holding page-zero data. References to symbols defined in that counter, or imported under it, then use direct addressing wherever the instruction has a direct mode, with no `<` needed. Their addresses can also be used as byte values, in 8-bit immediates (`ldaa #count`) and `FCB` data.

The object file format doesn't change, and a module that doesn't use `DPAGE` assembles exactly as before.

2 Background: how it works today

2.1 Choosing the addressing mode

For an instruction with both direct and extended modes, `as01` uses direct addressing when the operand is an absolute address that is known in the first pass and lies in `$00`–`$FF` (`as/mops.c`). Forward references and relocatable addresses always use extended addressing. A `<` before the operand forces direct addressing and a `>` forces extended.

The read-modify-write instructions (`NEG`, `COM`, `LSR`, `ROR`, `ASR`, `ASL`, `ROL`, `DEC`, `INC`, `TST`, `CLR`) and `JMP` have no direct mode on the MC6801, so they always use extended (or indexed) addressing.

2.2 Forced direct addressing of relocatable symbols

`<` already works with relocatable and imported symbols. `as01` emits the direct-mode opcode with a one-byte relocation, which the object format supports (a "relocation size" record selects 1, 2 or 4 bytes), and `ld01` fills in the low byte of the final address:

```
00005 0000   96     00> start   ldaa    <counter        ; imported
00006 0002   D6     00>         ldab    <local          ; in this module

SEG 0
: 96
RELSZ 1
DB counter
: d6
DB SEG3
```

Linked with the variables in page zero, this is correct: `96 81` and `D6 80`. Linked with them anywhere else, it is silently wrong. With `-D 0x5000`, `ld01` writes `96 04` and `D6 00`, the low bytes of `$5004` and `$5000`, and reports nothing: the program reads `$0004` and `$0000`.

2.3 What else makes one-byte relocations

Only two constructs produce one-byte relocations today:

Source	Meaning	Correct only if the value is
`ldaa <sym`	direct address	`$00`–`$FF`
`ldaa sym,x` (relocatable `sym`)	indexed offset	`$00`–`$FF`

Relocatable values in 8-bit immediates (`ldaa #sym`) and data (`fcb sym`) are rejected by `as01` ("absolute expression required"), and so are branches to imported labels ("non-imported expression required"). So a check that every one-byte relocation lands in `$00`–`$FF` can't reject any link that works today.

2.4 Imports carry a location counter

An `IMPORT` records the location counter that is current when it is read, and `ld01` rejects a link where the defining module exports the symbol from a different counter ("redefined with different segment or relocatability", then "undefined symbol"). Programs therefore already import a data symbol under its data counter:

```
        LOC     3
        IMPORT  __data_start, __data_end
        LOC     0
```

This design builds on that: an importing module knows which counter an imported symbol belongs to, and so whether it is in page zero.

3 Goals and non-goals

3.1 Goals

- Direct addressing for page-zero variables shared between modules and placed by a linker script, without writing `<` at every use.
- Page-zero addresses usable wherever a byte is expected: 8-bit immediates and `FCB` data.
- A link that puts such a variable outside page zero fails with a clear message, instead of producing wrong code.
- No change for modules that don't use the new directive, and no change to the object file format.

3.2 Non-goals

- Moving the direct page: the MC6801's direct page is fixed at `$00` (unlike the 6809's, which `as01`'s `dirpag` variable was written for).
- Direct modes the MC6801 doesn't have, such as `inc` or `clr` of a page-zero address: these stay extended.
- Choosing direct addressing for forward references, which are treated like forward references to `EQU` symbols (see Forward references).
- Placing direct-page counters automatically, or recording `DPAGE` in the object file: the program's linker script places direct-page counters, as it places any other memory.
- Warning about page-zero variables in the I/O registers (`$00`–`$1F`): a `MEMORY` region that starts at `$80` keeps them out.

4 The DPAGE directive

```
        DPAGE   4               ; location counter 4 is in page zero
```

`DPAGE n` declares that location counter `n` (0 to 15) holds data in page zero. It can be given for several counters, applies from where it appears to the end of the module, and should come before any use of the counter, usually at the top of the module or in an `INCLUDE` file that the modules share.

After `DPAGE n`, an operand whose value is relocatable *in counter n* uses direct addressing, with a one-byte relocation, if:

- the instruction has a direct mode (not `NEG`..`CLR` or `JMP`);
- the operand has no `>`; and
- the symbol is already defined or imported when the line is assembled.

That covers a symbol defined by a label, `COMM` or `EQU` in counter `n` earlier in the module, a symbol imported while counter `n` was current, and expressions such as `count+1` that are relocatable in counter `n`. Everything else is unchanged:

Operand	Without DPAGE	With DPAGE 4, count in counter 4
ldaa count	B6 hhll extended	96 ll direct
std count	FD hhll	DD ll
ldaa >count	B6 hhll	B6 hhll (forced extended)
inc count	7C hhll	7C hhll (no direct INC)
ldx #count	CE hhll	CE hhll (16-bit immediate)
ldaa #count	error	86 ll (8-bit immediate)
fdb count	2-byte address	2-byte address
fcb count	error	1-byte address
ldaa count, before count	B6 hhll	B6 hhll (forward reference)

A module that exports page-zero variables and the modules that import them must agree on the counter, which the existing import rule already requires.

4.1 Forward references

Forward references to direct-page symbols are treated exactly like forward references to EQU symbols. as01 decides an instruction's size in the first pass; a symbol defined later in the module isn't known yet, so a reference to it uses extended addressing, as for an absolute address defined later with EQU. To get direct addressing, define or import page-zero variables before the code that uses them, or write <. This keeps the choice the same in every pass, so it can't cause phase errors.

As with EQU, a forward reference is fine where the size doesn't depend on it: ldaa #count and fcb count are one byte whatever count turns out to be, so they accept a direct-page symbol defined later in the module.

4.2 Byte values

The address of a direct-page symbol fits in a byte once linked, so as01 accepts it where a byte value is expected, which today requires an absolute expression:

```
        ldaa    #count          ; 86 ll: the address of count
        cmpb    #count+1        ; C1 ll
        fcb     count, flags    ; two 1-byte addresses
```

Each is written with a one-byte relocation, which ld01 checks like a direct address. An expression that is relocatable in any other counter is still an error, as today.

4.3 Choosing a counter

Any counter can be used, but it must be one whose contents the linker script puts in page zero. Two common cases:

- **A separate counter**, such as LOC 4, for a program whose other data is elsewhere (an MC-10 program in external RAM, with a few variables in internal RAM). The script places LOC4 and COMMON4 in page zero.

- **The data counter itself**, such as `DPAGE 3`, for a program whose RAM is all internal, like the ROM example in the user's manual.

Common blocks (`COMM`) in a direct-page counter are direct-page symbols too, so the script must place the counter's `COMMONn` input as well as its `LOCn`.

On the MC-10, the ROM and BASIC use internal RAM for their own variables, so a program that returns to BASIC must check which page-zero bytes are free before using them.

5 Linker checks

When `ld01` applies a one-byte relocation, the final value must fit:

- an absolute one-byte relocation (a direct address, an indexed offset, an 8-bit immediate or `FCB` data) must be `$00`–`$FF`;
- a PC-relative one-byte relocation, if one ever appears, must be `-128`–`+127`.

Otherwise the link fails, as for an undefined symbol, with a message that names the module, the target and the address of the byte:

```
ld01: main.o: count ($5004) is not in page zero ($00-$ff), at $e001
ld01: crt0.o: LOC3+$0002 ($5002) is not in page zero ($00-$ff), at $e00c
```

The check needs no new information: `ld01` already knows the relocation size, the symbol and the final address. It runs whether or not the module used `DPAGE`, so it also catches `<` and indexed offsets. The object file doesn't record which counters are direct-page, so the check is made for each relocation rather than for a section as a whole; that catches every misplaced reference, and names it.

`ld01` doesn't warn about direct-page variables in the MC6801's I/O registers (`$00`–`$1F`): placing them is the linker script's job, and a `MEMORY` region starting at `$80` keeps them out.

No change is needed in the linker script language. A script places direct-page counters like any other input, and `ASSERT` can check a region's size:

```
MEMORY
{
    ROM (rx) : ORIGIN = 0xE000, LENGTH = 0x1FF0
    ZP  (rw) : ORIGIN = 0x0080, LENGTH = 128
    RAM (rw) : ORIGIN = 0x4000, LENGTH = 0x1000
}

SECTIONS
{
    .zpage (NOLOAD) : { *(LOC4) *(COMMON4) } > ZP
    ...
}
```

Without a script, the default layout puts every data counter after the text, so direct-page references fail the check. The message tells the user what is wrong, and `ld01 -P` prints the default script to start from.

6 Example

The ROM example in the user's manual keeps all its variables in the MC6801's internal RAM (`$80`–`$FF`). With `DPAGE 3`, and the variables declared before the code, its references to them become direct:

```
; crt0.s
        DPAGE   3               ; all of this program's data is in page zero
        LOC     3
        IMPORT  __data_start, __data_end
src     COMM    2               ; copy pointers
dst     COMM    2
        LOC     0
        ...
copy    ldx     src             ; DE src, was FE 0085
        ldaa    0,x
        inx
        stx     src             ; DF src, was FF 0085
        ...

; prog.s
        DPAGE   3
        LOC     3
pattern fcb     $55
count   COMM    1
        LOC     0
main    ldaa    pattern         ; 96 80, was B6 0080
loop    staa    PORT1
        coma
        inc     count           ; 7C 0089: INC has no direct mode
        bra     loop
```

The six `LDX`/`STX` references to `src` and `dst` in `crt0.s` and the `ldaa pattern` in `prog.s` each lose a byte and a cycle: the program's code shrinks from 63 bytes to 56, and the copy loop runs four cycles faster per byte. The linker script is unchanged, since it already puts counter 3 and its common blocks in internal RAM.

7 Implementation

7.1 1. ld01: check one-byte relocations

In `ld/writex86.c`, where `linkmod()` applies offset and symbol relocations, check the value when `relocsize` is 1, before writing it. Report errors with a new function in `ld/io.c`, alongside `size_error()` and `interseg()`, that names the module (and archive member), the symbol or counter, the value and the address. Relocatable output (`-r`) has no final addresses and isn't checked.

as01 must check too. For an address in one of the module's own counters, the object file holds the offset in the counter, written in one byte for a direct address, so an offset over $FF would be truncated before ld01 sees it; and a forced direct absolute address (ldaa <$1234) is truncated the same way. So when as01 uses direct addressing for an operand that is absolute or relocatable in the module (not imported), the value must fit in a byte, or it reports address out of bounds, as it already does for indexed offsets.

This phase stands alone as a bug fix, and could be released first.

7.2 2. as01: DPAGE

- Add DPAGE to the directive table (as/keywords.c) with a handler in as/pops.c that reads a counter number (0 to 15, or an error) and sets a bit in a 16-bit mask. Clear the mask at the start of each pass, so every pass sees the same declarations.
- In doaltind() (as/mops.c), choose direct addressing also when the operand is relocatable (RELBIT), not a forward reference, and its counter (data & SEGM) is in the mask, for instructions that have a direct mode and no >. genobjadr() already emits the one-byte relocation for a direct operand.
- A forward reference can't be recognized by FORBIT: in the normal two passes, IMPORT and COMM set it on every symbol they declare, and every reference sets it on a symbol that isn't a label. Instead, each symbol records whether it was used in the first pass before it was defined, imported or declared COMM (a new early field in struct sym_s), and an operand is a forward reference if any symbol in it was. The record is made in the first pass and never cleared, so every pass makes the same choice.
- In mimmed(), accept an 8-bit immediate that is relocatable in a direct-page counter instead of requiring it to be absolute (chkabs()). The operand is already written with genobjadr(), so it gets a one-byte relocation.
- FCB keeps only byte values (databuf.fcbuf), unlike FDB, which keeps full addresses so that they can be relocated. Keep FCB entries as addresses too when one is relocatable in a direct-page counter, and write each with genobjadr() and a size of 1, as FDB does with 2. -b binary output still requires absolute values.
- List DPAGE in the listing like other directives.

7.3 3. Documentation

- as01(1): the DPAGE directive, the addressing rules, and byte values of direct-page symbols in immediates and FCB.
- ld01(1): the new error, under DIAGNOSTICS.
- User's manual: the addressing section (replacing "relocatable addresses always use extended addressing"), the linker scripts chapter (replacing the "Page zero" pitfall), and the ROM example using DPAGE 3.

8 Testing

The regression tests now run programs on an MC6801 simulator, so the new code generation can be checked by running it, not just by its bytes.

- **as01:** listings and objects for a module using `DPAGE`: direct opcodes and one-byte relocations for symbols defined earlier, imported under the counter, and in expressions; extended for `>`, forward references, `NEG`..`CLR` and `JMP`; one-byte relocations for 8-bit immediates and `FCB` data, including forward references; unchanged 16-bit immediates and `FDB`; errors for a bad counter number and for byte values relocatable in other counters.
- **ld01:** the new error for a symbol relocation, an offset relocation, an indexed offset, an 8-bit immediate and `FCB` data out of range, each with its message and exit status; and success when the counter is in page zero.
- **Running:** the ROM example linked with `DPAGE 3`, run on the simulator, must leave the same RAM contents as the current version, with smaller code (checked in the `-M` map).
- **Compatibility:** every existing test's output must stay the same.

9 Phases

Phase	Work	Release
1	`ld01` range check (and `as01`'s, for absolute and local addresses), tests, `ld01(1)`	1.2.4. **Done.**
2	`as01 DPAGE`, tests	**Done.**
3	Documentation and the manual's ROM example	new feature (1.3.0). **Done.** The manual's examples are assembled and linked by `make check`, and the ROM example is run on the simulator.

10 Decisions

These questions were settled when the design was reviewed:

1. **The directive** is `DPAGE n`, one declaration per location counter, rather than an option on `LOC` or a per-symbol form such as `.importzp` in some 6502 assemblers. It fits how `as01` and `ld01` already treat imports and placement.
2. **Forward references** are treated like forward references to `EQU` symbols: extended addressing for instructions, and accepted where the size doesn't depend on the symbol (8-bit immediates and `FCB`). Revised after 2.0: `as01` now makes at least three passes, and more while instructions change size, so forward references to direct-page symbols, and to `EQU` symbols in page zero, get direct addressing too; the "used early" record is gone.
3. **Byte values:** `fcb count` and `ldaa #count` are allowed for direct-page symbols, with one-byte relocations that `ld01` checks.
4. **The object file** doesn't record `DPAGE`. The format stays as it is, and `ld01` checks each one-byte relocation instead of placing or checking whole sections.
5. **I/O registers:** `ld01` doesn't warn about direct-page variables at `$00`–`$1F`; the linker script decides where they go.